

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO TUẦN 2
MÔN THỰC TẬP CƠ SỞ

**Đề tài: Nghiên cứu, đánh giá và triển khai
mô hình Deep Learning tối ưu cho hệ thống phân loại rác thải**

Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Sinh viên thực hiện:	Nguyễn Văn Gia Bảo
Mã sinh viên:	B23DCCN079

Hà Nội - 2026

MỤC LỤC

1. Mục tiêu công việc trong tuần	3
2. Nội dung công việc đã thực hiện	3
2.1 Cài đặt môi trường	3
2.2 Thu thập và khám phá tập dữ liệu (dataset)	3
2.3 Tìm hiểu PyTorch Dataset và DataLoader	3
2.4 Tiền xử lý ảnh	3
2.5 Chia dataset train/validation/test	4
3. Kết quả đạt được	4
4. Khó khăn gặp phải	5
5. Kế hoạch tuần tiếp theo	5
TÀI LIỆU THAM KHẢO	5

1. Mục tiêu công việc trong tuần

- Cài đặt môi trường phát triển đầy đủ
- Tải và khám phá dataset TrashNet
- Tìm hiểu PyTorch Dataset và DataLoader
- Viết code tiền xử lý ảnh: resize, normalize, augmentation
- Chia dataset thành tập train/validation/test theo tỉ lệ 70/15/15

2. Nội dung công việc đã thực hiện

2.1 Cài đặt môi trường

Em đã cài đặt đầy đủ các thư viện cần thiết cho dự án, bao gồm PyTorch 2.6.0, torchvision, Streamlit, OpenCV, scikit-learn, matplotlib, pandas và numpy.

2.2 Thu thập và khám phá tập dữ liệu (dataset)

Em đã chọn và tải tập dữ liệu TrashNet - tập dữ liệu công khai phổ biến cho bài toán phân loại rác thải [1]. Dataset gồm **2527 ảnh** thuộc **6 danh mục**: cardboard (bìa các tông), glass (thủy tinh), metal (kim loại), paper (giấy), plastic (nhựa) và trash (rác hỗn hợp).

2.3 Tìm hiểu PyTorch Dataset và DataLoader

Em đã tìm hiểu 2 thành phần quan trọng trong PyTorch để xử lý dữ liệu [2]:

Dataset là đối tượng quản lý toàn bộ dữ liệu - trỏ tới đường dẫn ảnh và gán nhãn tự động theo tên thư mục. Trong đề tài này, datasets.ImageFolder được sử dụng để tự động đọc ảnh và gán nhãn dựa vào cấu trúc thư mục của TrashNet.

DataLoader là đối tượng chia dataset thành các batch nhỏ (batch_size=32) để đưa vào model từng đợt thay vì đưa toàn bộ cùng lúc - giúp tiết kiệm bộ nhớ RAM/VRAM và ổn định quá trình huấn luyện.

2.4 Tiền xử lý ảnh

Em đã xây dựng pipeline tiền xử lý ảnh sử dụng transforms.Compose [2].

```

9 # Tiền xử lý ảnh cho tập train (có augmentation: mỗi ảnh đưa vào 1 lần, mỗi lần có thể là nguyên mẫu hoặc đã qua biến đổi)
10 train_transforms = transforms.Compose([ # transforms.Compose định dạng lại dữ liệu (ảnh)
11     transforms.Resize((224, 224)), # resize về 224x224
12     transforms.RandomHorizontalFlip(), # lật ảnh ngẫu nhiên
13     transforms.RandomRotation(10), # xoay ảnh ngẫu nhiên 10 độ
14     transforms.ColorJitter(brightness=0.2, contrast=0.2), # thay đổi độ sáng
15     transforms.ToTensor(), # chuyển ảnh thành tensor (ma trận số (R, G, B))
16     transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]) # chuẩn hóa theo chỉ số đặc trưng của ImageNet
17 ])
18
19 # Tiền xử lý ảnh cho tập val và test (không augmentation)
20 val_test_transforms = transforms.Compose([
21     transforms.Resize((224, 224)),
22     transforms.ToTensor(),
23     transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
24 ])

```

2.5 Chia dataset train/validation/test

Em đã sử dụng **train_test_split** với tham số **stratify** để đảm bảo các nhãn được phân bố đều vào cả 3 tập [3], kết hợp với **Subset** để áp dụng transform phù hợp cho từng tập:

- **Train:** 1.768 ảnh (70%) — dùng train_transforms có augmentation
- **Validation:** 379 ảnh (15%) — dùng val_test_transforms không augmentation
- **Test:** 380 ảnh (15%) — dùng val_test_transforms không augmentation

Tham số **random_state=42** được cố định để đảm bảo kết quả chia dữ liệu nhất quán mỗi lần chạy.

```

26 # Load dataset 2 lần: train dùng train_transforms, val/test dùng val_test_transforms, dán nhãn cho ảnh
27 train_dataset = datasets.ImageFolder(DATASET_PATH, transform=train_transforms)
28 val_test_dataset = datasets.ImageFolder(DATASET_PATH, transform=val_test_transforms)
29
30 # Lấy danh sách index và nhãn
31 total = len(train_dataset) # số ảnh trong dataset: 2527
32 indices = list(range(total))
33 labels = [train_dataset.targets[i] for i in indices] # lấy nhãn từng ảnh theo index
34
35 # Chia train/val/test theo tỉ lệ 70/15/15, đảm bảo các nhãn chia đều (stratify)
36 train_indices, val_test_indices = train_test_split(
37     indices, test_size=0.3, stratify=labels, random_state=42
38 ) # giá trị 2 biến là tập các index đã được xáo trộn ngẫu nhiên theo tỷ lệ 7 train và 3 test. Cố định cách xáo để so sánh được 3 model sau này
39 val_indices, test_indices = train_test_split(
40     val_test_indices, test_size=0.5, random_state=42
41 )
42
43 # Tạo Subset từ index đã chia
44 train_set = Subset(train_dataset, train_indices)
45 val_set = Subset(val_test_dataset, val_indices)
46 test_set = Subset(val_test_dataset, test_indices)

```

3. Kết quả đạt được

- Cài đặt xong môi trường phát triển
- Tải và kiểm tra dataset TrashNet đầy đủ 6 class, 2.527 ảnh
- Viết và chạy thành công file data_loader.py

4. Khó khăn gặp phải

- Lần đầu tiếp xúc với PyTorch nên một số khái niệm như Dataset, DataLoader, Subset còn khá mới, cần thêm thời gian tìm hiểu trước khi thực hành
- Cần tìm hiểu và đánh giá nhiều bộ dữ liệu khác nhau để chọn được dataset phù hợp với bài toán phân loại rác thải, cuối cùng chọn TrashNet vì có đủ 6 danh mục rác thải phổ biến, ảnh đã được resize sẵn và có cấu trúc thư mục rõ ràng
- Cần xác định định dạng dataset phù hợp để PyTorch có thể đọc được — sau khi tìm hiểu đã sử dụng cấu trúc thư mục theo từng class để datasets.ImageFolder tự động gán nhãn
- Cần lựa chọn cách chia dữ liệu phù hợp để đảm bảo các nhãn được phân bố đều vào cả 3 tập train/validation/test

5. Kế hoạch tuần tiếp theo

- Tìm hiểu fine-tuning và freeze layer trong Transfer Learning
- Load VGG16 pretrained từ torchvision
- Chỉnh lớp cuối cho phù hợp 6 class của TrashNet
- Viết training loop: forward, backward, optimizer, scheduler
- Huấn luyện VGG16, đánh giá kết quả trên tập validation

TÀI LIỆU THAM KHẢO

- [1] G. Thung and C. Yang, "TrashNet: A dataset for trash classification," Stanford University, <https://github.com/garythung/trashnet>, 2016.
- [2] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.